

4

プログラミング概論I (オンライン授業)



担当: 中山 功一 (7号館3階308室)

メールアドレス: knakayama@is.saga-u.ac.jp

326

オープニングテスト

前回, 習ったことを復習するために,
オープニングテストを行います

問題ごとに動画を止めて, 答えをメモしましょう.

最後に, 答えを動画で表示しますので,
自分が理解しているか, 確認しましょう.

オープニング テスト

問1~4: 次のプログラムの出力を答えなさい

```
int a = 2, b = 4, c;  
c = 3 % a * 2;  
cout << c;    //問1  
b = (a + b) % 9;  
cout << b;    //問2  
a *= a + 2;  
cout << a;    //問3  
a = b = c = 2;  
cout << a;    //問4
```

<選択肢>

- | | |
|-----|-----|
| ① 1 | ⑥ 6 |
| ② 2 | ⑦ 7 |
| ③ 3 | ⑧ 8 |
| ④ 4 | ⑨ 9 |
| ⑤ 5 | ⑩ 0 |

328

```
int suuji;  
cin >> suuji;  
switch(suuji){  
    case 1:  
        cout << "一";  
    case 2:  
        cout << "二";  
        break;  
    default:  
        cout << "他";  
        break;  
}
```

問5: suuji=1のとき
問6: suuji=2のとき
問7: suuji=3のとき
の画面表示は?

<選択肢>

- | | |
|-------|-------|
| ① 一二他 | ④ 一二 |
| ② 二他 | ⑤ 一 |
| ③ 二 | ⑥ 三 |
| ⑦ 一他 | ⑧ その他 |

329

```
int a = 2, b=3;  
if(a%2!=2){  
    cout << 1;  
    if(b%2==0){  
        cout << 2;  
    }  
}else{  
    cout << 3;  
}
```

問8: 次のプログラムの出力を
答えなさい
(選択肢のコンマは無視すること)

<選択肢>

- | |
|-----------|
| ① 1 |
| ② 2 |
| ③ 3 |
| ④ 1, 2 |
| ⑤ 2, 3 |
| ⑥ 1, 3 |
| ⑦ 1, 2, 3 |
| ⑧ その他 |

330

```
int score;
cin >> score;
if (score >= 80) {
    cout << "優";
}
if (score >= 70) {
    cout << "良";
}
else if (score >= 60) {
    cout << "可";
}
if (60 > score) {
    cout << "不";
}
}
```

以下の数字を入力した
ときの画面表示は？
(空欄右寄せ)

入力 画面出力

85 : 問【9】

75 : 問【10】

65 : 問【11】

55 : 問【12】

331

オープニング テスト

五十音順に3500万語が収録されている電子辞書がある。目的の単語を二分探索で調べる場合、平均して約【問13】回の探索が必要である

<選択肢>

①5, ②10, ③15, ④20, ⑤25, ⑥30, ⑦35,
⑧40, ⑨50, ⑩60

332

答え合わせ

全ての答えをメモしましたか？

解答

- | | |
|-----|-------------------|
| 【1】 | ② 1 * 2 |
| 【2】 | ⑥ 6 % 9 |
| 【3】 | ⑧ 2 * = 4 |
| 【4】 | ② a = 2 |
| 【5】 | ④ breakが「二」までないため |
| 【6】 | ③ |
| 【7】 | ① 「他」と表示される |
| 【8】 | ① 「1」と表示される |

解答

- 【9】 優良
※ elseの有無に注意
80点以上はtrue
70点以上もtrue
(elseがないので別のif文)
- 【10】 良
- 【11】 可
- 【12】 不
- 【13】 ⑤ 「25回」10回で1000(k), 20回で100万(M), 25回で32 × 100万

関数

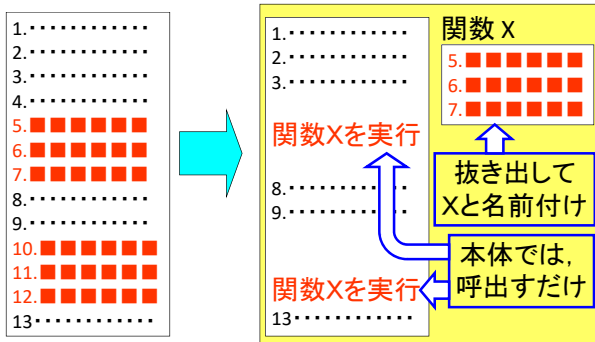
関数

- 複数の文で実現される
ひと固まりの機能をまとめたもの
 (メイン関数も関数の一つ)
 (ファンクション／プロシージャとも呼ぶ)
- 関数の目的
 - 複雑なプログラムを分割して作成する
 - 内部アルゴリズムの理解が不要になる**注: 数学の関数とは違う**

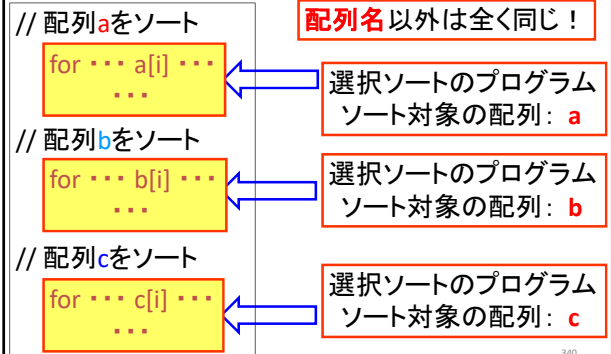
338

関数

- イメージ: 複数命令をまとめたもの
- 同様の処理が何回も出る場合便利

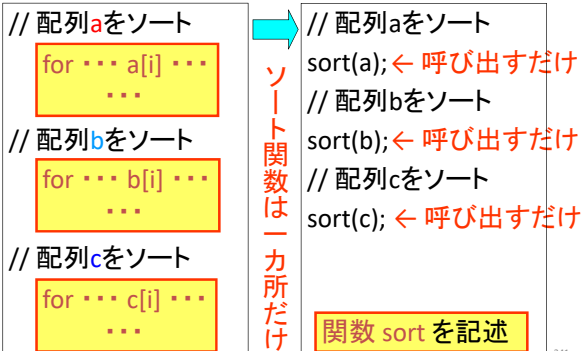


具体例: 複数の配列をソートする



340

処理を関数として独立させる



341

関数の関係(親子のアナロジー)

関数Aが関数Bを呼び出し、
関数Bが関数Cを呼び出す場合

- 関数Aからみて、
関数B: 子関数, 関数C: 孫関数
- 関数Bからみて、
関数A: 親関数, 関数C: 子関数

親子はいつでも逆転できる

- 親や自分自身も、子関数として呼び出せる

関数とのデータのやり取り

<引数:親→子へ渡すデータ>

- 引数なし: 子関数にデータを与えない
- 引数あり: 子関数にデータを与える
 - 親が渡す値(変数の中身) ⇒ **実**引数
 - 子が受け取る変数(コピー) ⇒ **仮**引数

1対1
で対応

<戻り値:子→親へ戻すデータ>

- 戻り値なし: 親にデータに戻さない(やっといて)
- 戻り値あり: 親にデータに戻す(結果を教えて)

関数の書き方

- 関数の**プロトタイプ宣言**「関数を使うぞ！」
プログラムの冒頭(メインプログラムの前)に
「戻り値の型・関数名・引数の型」を宣言する
- 関数の**定義**「関数の中身はこうだ！」
「関数の中身(ソースコード)」を記述
(プロトタイプ宣言の後で書くこと)
- 関数の**呼び出し**「関数を実行しろ！」
親関数の中で子関数を呼び出し、実行する

344

関数の使用例

- 関数の**プロトタイプ宣言**

戻り値の型 関数名 引数名
int max(int array[10]);
引数の型

- 関数の**定義**

```
int max( int array[10]){
    /*関数の中身*/
}
```

- 関数の**呼び出し**

```
int Num = max(array[10]);
```

345

1. 関数のプロトタイプ宣言

- プロトタイプ宣言に含まれる情報
 - 戻り値のデータ型(1個または無し: void)
 - 関数名
 - 仮引数の名前とデータ型(0個以上)
- 文法
戻り値の型 関数名(仮引数の型 仮引数名);
- 例

```
int total(int array[N]);
```



```
void getMatrix();
```



```
void exchangeRow(int row1, int row2);
```

関数定義と同一

346

2. 関数の定義

- 関数の定義に含まれる情報
 - 戻り値のデータ型(1個または無し: void)
 - 関数名
 - 仮引数の名前とデータ型(0個以上)
 - 関数の本体(ソースコード)**
- 文法

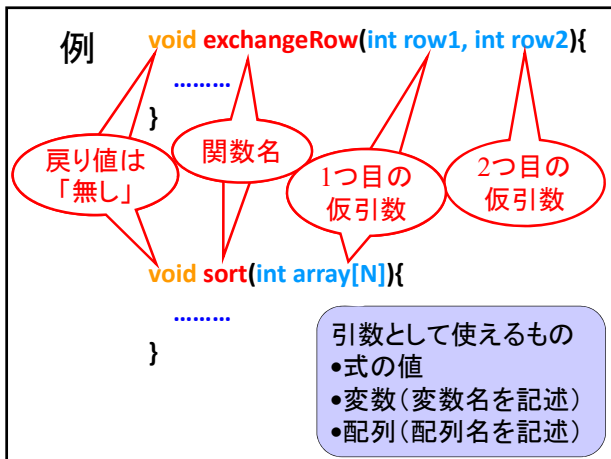
戻り値の型 関数名(仮引数の型 仮引数名){
 ソースコード
 (変数定義, 文)
}

プロトタイプ宣言
と同一

※ 仮引数が複数ある場合は「,」で区切る

347

```
//配列matrix( N行M列)に値を入力する
void getMatrix(){
    // 各行について以下の処理を繰り返す
    for (int i=0; i<N; i++){
        // 各列について要素の値を入力する
        for (int j=0; j<M; j++) {
            cout << "行列の" << i << "行" << j <<
            "列要素を入力してください";
            cin >> matrix[i][j];
        }
    }
}
```



3. 関数の呼び出し

- 文法

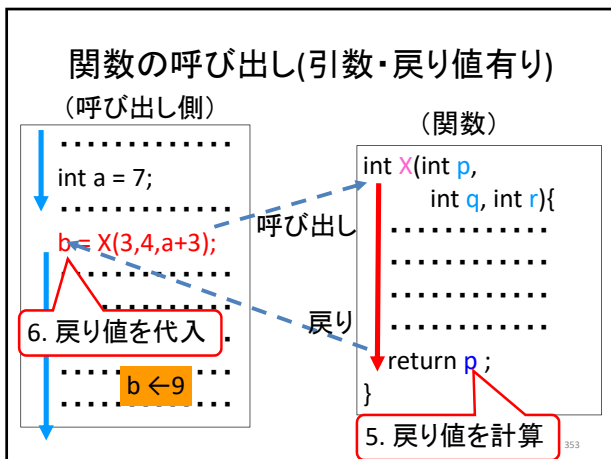
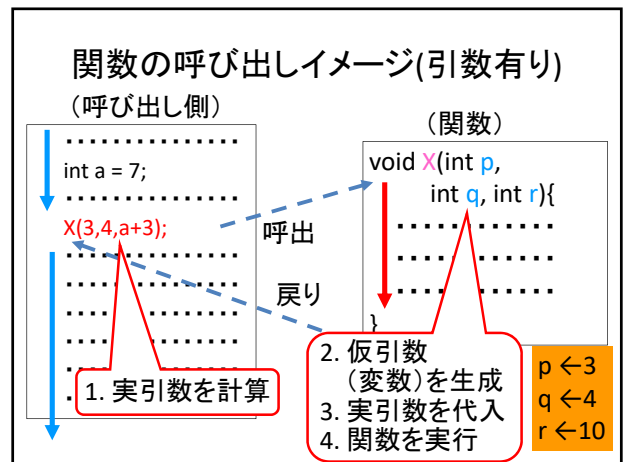
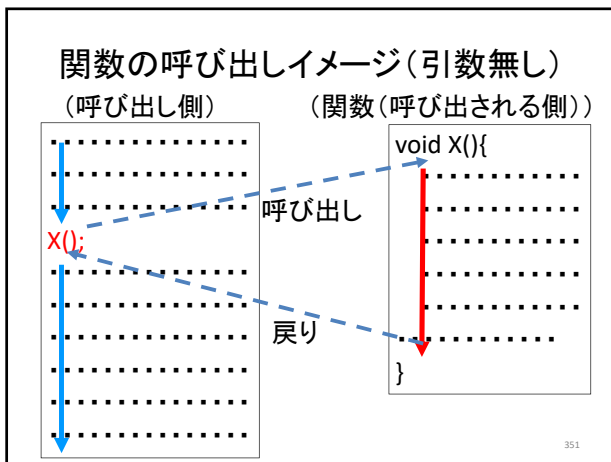
```
関数名 (実引数);           // 戻り値なし
変数 = 関数名 (実引数);    // 戻り値あり
```

例:

```
getMatrix();               exchangeRow(2, 4);
sum = total(matrix);       average = total(matrix) / N;
if (total(matrix) == 0) { /* 略 */ }
```

戻り値を
変数sumに格納

戻り値を直接
式の中で使う



関数の終了

- 関数は、**return**文により終了する
 - 戻り値なし: **return;**
 - 戻り値あり: **return 戻り値;**
- 関数末尾まで到達すると自動的にreturn
(=関数末尾のreturn文は省略可能)

関数(引数, 戻り値)のまとめ

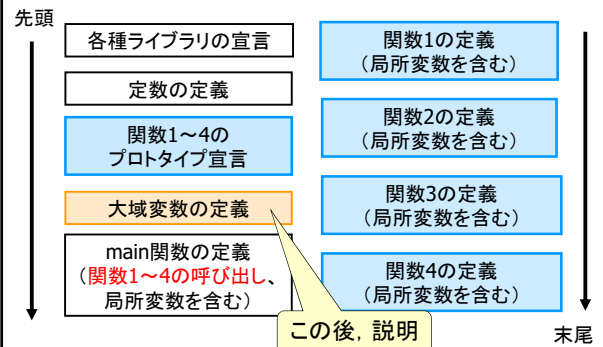
<プロトタイプ宣言>

		戻り値	
		あり(int)	なし
引数	あり(int data)	int func(int data)	void func(int data)
	なし	int func(void)	void func(void)

<親関数内での呼出し>

		戻り値	
		あり(int)	なし
引数	あり(int data)	A = func(2)	func(2)
	なし	A = func()	func()

関数と大域変数を含むプログラムの基本構造



関数を使う際の注意点

仮引数の値は実引数に反映されない

```
void main(void){
    double x = 0.0;
    cout << "xの初期値=" << x << endl;
    increment(x);
    cout << "xの最終値=" << x << endl;
}
// サブルーチン
void increment(double value){
    value ++;
```

実行結果
xの初期値=0.0
xの最終値=0.0

例外となる引数: 配列

- 配列を実引数として関数に渡せる
 - 関数の中で配列の要素を参照できる
 - 関数の中で配列の要素を変更できる
- ※ 配列以外の仮引数を変更しても、対応する実引数には反映されない

配列の仮引数だけは、実引数に反映される

```
void main(void){
    int table[10];
    initialize(table);
    for(int i=0; i<10; i++){
        cout << table[i] << ", ";
    }
}
// サブルーチン
void initialize(int array[10]){
    for(int i=0; i<10; i++){
        array[i] = 2*i+1;
```

実行結果
1, 3, 5, 7, 9, 11,
13, 15, 17, 19,

関数の動きを理解しよう

```
m01: void main(){
m02:     int x=2, y=3, ans;
m03:     ans = Shisu(x, y);
m04:     cout << "xのy乗は" << ans << "¥n";
m05:     ans = ans + Shisu(y, x);
m06:     cout << "xのy乗 + yのx乗は"
        << ans << "¥n";
m07: }
```

関数の動きを理解しよう

```
c1-01: int Shisu (int c_tei, int c_shisu){
c1-02:     int i, c_ans = 1;
c1-03:     for( i = 0 ; i < c_shisu; i++){
c1-04:         c_ans *= c_tei;
c1-05:     }
c1-06:     return c_ans;
c1-07: }
```

361

トレース表を書きなさい

362

回答用紙を
交換してください

間違っている部分に
赤ペンで正解を
記入してください

行数	変数	メイン関数			サブルーチン1				サブルーチン2			
		HE	HE	HE	HE	HE	HE	HE	HE	HE	HE	HE
m02	x	2	3	-								
m03 > c1-01	y	2	3	-	2	3						
c1-02		2	3	-	2	3	1	-				
c1-03		2	3	-	2	3	1	0				
c1-04		2	3	-	2	3	2	0				
c1-03		2	3	-	2	3	2	1				
c1-04		2	3	-	2	3	4	1				
c1-03		2	3	-	2	3	4	2				
c1-04		2	3	-	2	3	8	2				
c1-03		2	3	-	2	3	8	3				
m03 < c1-06		2	3	8	2	3	8	3				
m05 > c2-01		2	3	8					3	2		
c2-02		2	3	8					3	2	1	-
c2-03		2	3	8					3	2	1	0
c2-04		2	3	8					3	2	3	0
c2-03		2	3	8					3	2	3	1
c2-04		2	3	8					3	2	9	1
c2-03		2	3	8					3	2	9	2
m05 < c2-06		2	3	17					3	2	9	2

関数(引数, 戻り値)の例

		戻り値	
		あり	なし
引数	あり	r = sin(angle); printf("hello");	
	なし	c = getchar(); Beep();	

引数 : 関数に渡すデータ

呼ばれた関数に受け取る箱が必要

戻り値 : 関数から戻ってくるデータ

呼んだ関数に受け取る箱が必要

364

引数を渡す場合の例

・プロトタイプ宣言

```
void function(int data, double value);
```

・呼び出す関数

```
function(32, 54.3);
```

・呼び出される関数

```
void function(int data, double value){
/*略*/
}
```

※ 順番を入れ替えてはいけない

渡す値が2つ

受け取る箱
(変数)が2つ

戻り値がある場合の例

・プロトタイプ宣言

```
int function(void);
```

・呼び出す関数

```
data = function();
```

受け取る箱(変数)

・呼び出される関数

```
int function(void){
int X;
/*途中省略*/
return X;
}
```

戻すデータ

ライブラリ: 先人が作った関数が多数

- 自分で作成した関数
使う際にはプロトタイプ宣言も自分で
- ライブラリを使用
ライブラリのインクルードが必要
プロトタイプ宣言もプログラム内では不要
(ライブラリ内で勝手にやってくれる)

367

ライブラリを用いるプログラムの基本構造

```
#include <iostream>
using namespace std;
#include <math.h>
```

各種ライブラリ
の宣言

main関数

```
void main(){
    // 数値演算ライブラリを呼び出せる
}
```

368

数値演算ライブラリ

ライブラリの使用を明示

– #include <math.h>

ライブラリ関数

- 角度の単位はラジアン ($360^\circ = 2\pi$ ラジアン)
- どの関数も、double型の値1つを引数とし、double型の値1つを返す

369

代表的なライブラリ関数

- sqrt : 平方根
- log : 対数 (底はe)
- log10 : 対数 (底は10)
- sin
- cos
- tan
- asin : $\sin^{-1}$
(sinの逆関数)
- acos : $\cos^{-1}$
(cosの逆関数)
- atan : $\tan^{-1}$
(tanの逆関数)

<呼び出し例>

```
#define PI 3.1416
double x, y, z, w;
x = sin( PI/2 );
y = log( (double)1000 );
z = acos( -0.5 );
w = sqrt( (double)256 );
```

370

標準ライブラリ関数の例: sqrt(x)

```
#include <iostream>
```

```
#include <math.h>
```

```
using namespace std;
```

```
void main(){
```

```
    double a, b=2.5;
```

引数

```
    a = sqrt(b);
```

```
    cout << b << "の平方根は" << a << "です¥n";
```

```
}
```

戻り値
を代入

371

標準ライブラリ関数の例: time(0)

```
#include <iostream>
```

```
#include <time.h> // #include <ctime>でもよい
```

```
using namespace std;
```

```
void main(){
```

```
    int t;
```

```
    t = time(0);
```

```
    cout << "1970年の元旦から" << t << "秒です¥n";
```

```
    t /= 60 * 60 * 24 * 365;
```

```
    cout << "1970年の元旦から" << t << "年です¥n";
```

```
}
```

372

標準ライブラリ関数の例: clock()

```
#include <iostream>
#include<time.h> // #include<ctime>でもよい
using namespace std;
int main(void){
    double start, end;
    long i;
    start = clock();
    for (i=0; i < 1000000000; i++){ };
    end = clock();
    cout << "空ループ1億回の時間"
         << (end - start) / CLOCKS_PER_SEC << "秒¥n";
}
```

373

標準ライブラリ関数の例: rand()

```
#include <iostream>
#include<stdlib.h>
using namespace std;
void main(){
    int i,t;
    double r;
    for(i = 0 ; i < 10 ; i++){
        t = rand() % 6 + 1;
        cout << "サイコロ値は" << t << "です¥n";
        r = rand() / (double)RAND_MAX;
        cout << "ランダム値は" << r << "です¥n";
    }
}
```

374

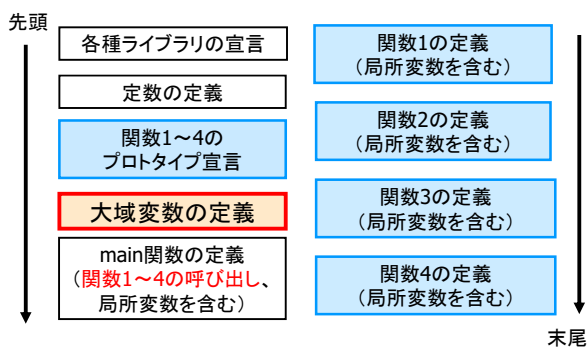
様々な変数

局所変数と大域変数

- **局所変数** (通常はこちら)
 - 定義した関数内のみでアクセスできる変数
 - 関数毎に独立に定義
 - 関数呼出時に生成, 関数終了時に消去
 - 異なる関数の局所変数は別の変数(変数名が同じでも)
- **大域変数** (通常は用いない)
 - どの関数からでもアクセスできる変数
 - プログラム開始時に生成, 終了時に消去
- 定義方法は同じ. 定義する場所が異なる

376

関数と大域変数を含むプログラムの基本構造



末尾

動的変数と静的変数

- **動的変数** (通常はこちら)
 - 関数を呼び出すごとに初期化される変数
 - 前回の処理結果は保持されない
 - 関数呼び出し時に生成, 関数終了時に消去
- **静的変数** (通常は用いない)
 - 呼び出すとプログラム終了まで値が保持される
 - 最初から最後まで, 同じメモリー領域で記憶
- 「static 型名 変数名」で定義すれば静的変数

378

変数へのメモリ割当

- 大域変数(グローバル変数)
 - プログラム開始時にメモリ確保
プログラム終了まで同じ場所
 - 静的記憶領域
- 局所変数(ローカル変数)
 - 関数呼出時にメモリ確保
関数終了時にメモリ解放
 - スタック領域(スタック構造とは意味が違う)
- コンパイラが、自動で手はずを整える

関数の再帰呼び出し

再帰呼び出し

- 自分で自分を呼び出す関数
 - 定義したら、何度でも呼び出せる
- 引数を変化させながら呼び出す
同じ引数だと無限ループになってしまう

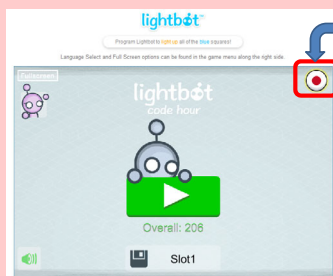
例:階乗の計算

$$\begin{aligned} N! &= N \times (N-1)! = N \times (N-1) \times (N-2)! \\ &= N \times (N-1) \times \cdots \times 2 \times 1 \end{aligned}$$

```
int Kaijo(int N){
    int i;
    if(N == 1){
        return 1;
    } else {
        i = N * Kaijo(N - 1);
        return i;
    }
}
```

382

ノートPC(スマホでもOK): Lightbotゲーム

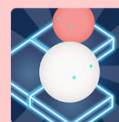


日本語に変更
「1 基本」
「2: プロシージャ
(関数)」
「3: ループ
(再起呼出)」

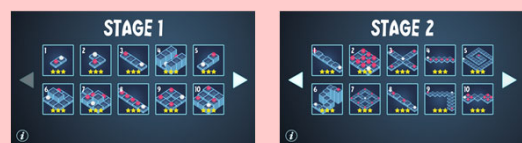


<https://lightbot.com/index.html>
※Flash版ではなくアプリ版を使うこと

スマホアプリ: Digital Puppet



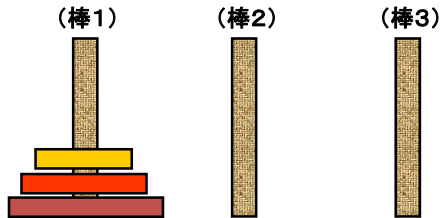
プログラミングパズル
(iPhone版/Android版)



384

パズルを解くのに使える

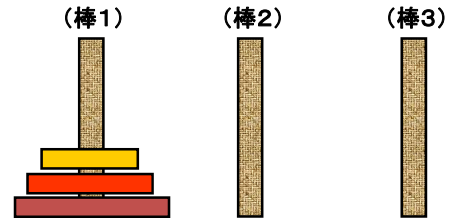
・「ハノイの塔」の問題



- 左端の円盤をすべて右端に移動する
- 円盤は1枚ずつ移動する
- 小さい円盤の上に大きい円盤を置いてはいけない

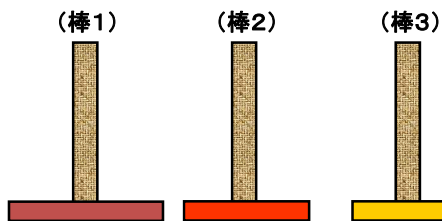
パズルを解くのに使える

・「ハノイの塔」の問題



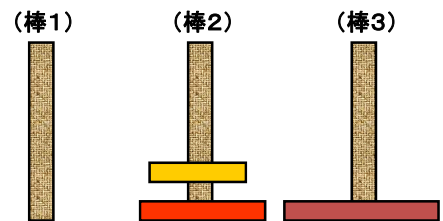
パズルを解くのに使える

・「ハノイの塔」の問題



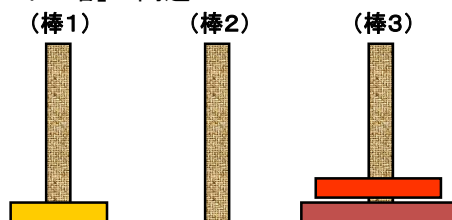
パズルを解くのに使える

・「ハノイの塔」の問題



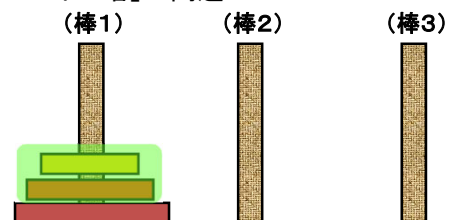
パズルを解くのに使える

・「ハノイの塔」の問題



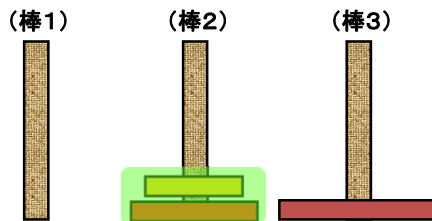
パズルを解くのに使える

・「ハノイの塔」の問題



パズルを解くのにも使える

・「ハノイの塔」の問題



再帰を使って解く

・ 考え方:

- 棒Xから棒Yへ、N枚の円盤を移動させる方法
 - ① 上から(N-1)の円盤を、棒Xから棒Zへ移動させる(退避させる)
 - ② 一番大きい円盤を、棒Xから棒Yへ移動させる
 - ③ 退避させた棒Zの(N-1)の円盤を、棒Zから棒Yへ移動させる

再帰的に解けば良い

```
void move(int x, int y, int n){
    if(n == 1) {
        cout <<"盤"<<n<<"を"<<x<<"から"<<y<<endl;
    } else {
        int tmp=1;
        if(x == 1 && y == 2) tmp = 3;
        if(x == 1 && y == 3) tmp = 2;
        if(x == 2 && y == 1) tmp = 3;
        if(x == 2 && y == 3) tmp = 1;
        if(x == 3 && y == 1) tmp = 2;
        if(x == 3 && y == 2) tmp = 1;
        move(x, tmp, n-1);
        cout <<"盤"<<n<<"を"<<x<<"から"<<y<<endl;
        move(tmp, y, n-1);
    }
}
```

待避場所を使う棒の番号をtmpに入れる

画面表示

```
盤1を1から3
盤2を1から2
盤1を3から2
盤3を1から3
盤1を2から1
盤2を2から3
盤1を1から3
```

クロージングテスト

今回、習ったことを復習するために、クロージングテストを行います

問題ごとに動画を止めて、答えをメモしましょう。

最後に、答えを動画で表示しますので、自分が理解しているか、確認しましょう。

クロージングテスト

問1～8 関数func()の正しい記述を選びなさい

プロトタイプ宣言		戻り値	
引数	あり(int data)	問1	問2
	なし	問3	問4
main関数での記述		戻り値	
引数	あり(int data)	問5	問6
	なし	問7	問8

<選択肢>

- ① A = func(2)
- ② int func(void)
- ③ func()
- ④ int func(int data)
- ⑤ func(2)
- ⑥ void func(void)
- ⑦ void func(int data)
- ⑧ A = func()

396

クロージング テスト

メイン関数から別の関数を呼び出すとき、メイン関数から与える値を【問9】、呼び出された関数が受け取る変数を【問10】と呼ぶ。これらの値は2個以上【問11】。また、0個【問12】。このとき、実引数と仮引数の個数は、【問13】

<選択肢>①仮引数, ②実引数, ③でもよい, ④ではいけない, ⑤必ず一致する必要がある, ⑥必ずしも一致しなくても良い,

397

クロージング テスト

標準ライブラリ関数を用いる場合は、【問14】で該当するライブラリを呼び出す必要がある。例えば、rand()関数を用いる場合は【問15】を、clock()関数を用いる場合は【問16】を、sqrt()関数を用いる場合は【問17】を呼び出す必要がある。標準ライブラリ関数を用いる場合は、プログラマは、その関数のプロトタイプ宣言をする必要が【問18】

<選択肢>①ある, ②ない, ③#define, ④#include, ⑤void, ⑥time.h, ⑦stdlib.h, ⑧stdio.h, ⑨math.h, ⑩ iostream

398

答え合わせ

全ての答えをメモしましたか？

解答

【1】	④	【9】	②	【14】	④
【2】	⑦	【10】	①	【15】	⑦
【3】	②	【11】	③	【16】	⑥
【4】	⑥	【12】	③	【17】	⑨
【5】	①	【13】	⑤	【18】	②
【6】	⑤				
【7】	⑧				
【8】	③				

お疲れさまでした

